MASTÈRE EXPERT EN DÉVELOPPEMENT ET DÉPLOIEMENT DE SOLUTIONS NUMÉRIQUES SÉCURISÉES · RNCP 40166









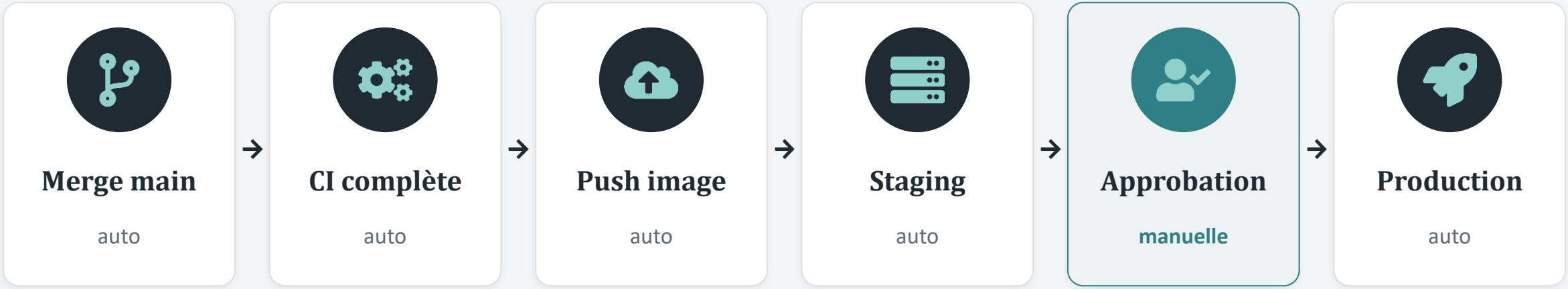
CI/CD

Déployer en production : Infrastructure as Code, release et conformité

JOUR 2 · Terraform, Ansible, GitLab CI, rollback, SBOM

Formateur : Alimou Diallo · 10/11/2026 · Bloc 5 – E52 · Sup de Vinci Rennes

Staging, approbation, production



Même image partout

L'artefact testé en staging est celui déployé en production.

Config par environnement

Variables et secrets séparés, jamais dans l'image.

Approbation tracée

Qui a validé, quand, quelle version : preuve d'audit.

Terraform : décrire l'infrastructure



```
resource "aws_instance" "app" {
  ami           = var.ami
  instance_type = "t3.small"
  key_name      = var.ssh_key
  tags = { Name = "backend-prod" }
}

output "public_ip" {
  value = aws_instance.app.public_ip
}
```

`terraform init`
Télécharge les providers

`terraform plan`
Prévisualise les changements

`terraform apply`
Crée ou modifie l'infra

`terraform destroy`
Détruit proprement

Exemple : adapter le provider à votre environnement

Terraform : état, variables et sécurité

```
variable "ami" { type = string }
variable "ssh_key" { type = string }

# terraform.tfvars  (hors Git)
ami     = "ami-0abc1234"
ssh_key = "deploy-key"

# Backend distant (state partagé)
terraform {
  backend "s3" { bucket = "tfstate-projet" }
}
```

Fichier d'état

Source de vérité de l'infra : à stocker en backend distant et verrouillé

Secrets

Variables sensibles via CI/CD, jamais dans Git ni dans le state en clair

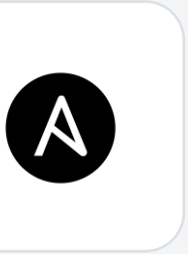
Code versionné

Modules réutilisables, revue de code du plan avant apply

Reproductible

Même code = même infrastructure, en staging comme en production

Ansible : configurer et déployer sans agent



Inventaire
Liste des serveurs cibles

Playbook
Tâches décrites en YAML

Idempotence
Relancer sans effet de bord

```
# inventories/prod
[app]
203.0.113.10 ansible_user=deploy

# deploy.yml
- hosts: app
  become: true
  tasks:
    - name: Installer Docker
      ansible.builtin.package:
        name: docker.io
        state: present
    - name: Lancer le backend
      community.docker.docker_container:
        name: backend
        image: "{{ image }}:{{ tag }}"
        state: started
        restart_policy: always
        published_ports: ["8080:8080"]
```

Exécuter le playbook et protéger les secrets

En pratique

```
# Vérifier la connectivité
ansible -i inventories/prod app -m ping

# Simuler sans rien modifier
ansible-playbook -i inventories/prod deploy.yml \
  --check --diff -e tag=1.0.0

# Déployer
ansible-playbook -i inventories/prod deploy.yml \
  -e tag=1.0.0
```

Ansible Vault

```
ansible-vault encrypt \
  group_vars/prod/secrets.yml

ansible-playbook deploy.yml \
  --vault-password-file \
  $VAULT_FILE
```

Mot de passe vault : credential du CI, jamais dans Git.

Terraform puis Ansible : la chaîne complète



Le pont entre les deux outils

```
terraform apply -auto-approve
IP=$(terraform output -raw public_ip)
echo "[app]\n$IP ansible_user=deploy" > inventories/prod
ansible-playbook -i inventories/prod deploy.yml -e tag=$TAG
```

GitLab CI : le même pipeline en .gitlab-ci.yml



```yaml
stages: [test, package, staging, production]

build-image:
  stage: package
  image: docker:27
  services: [docker:27-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

deploy-prod:
  stage: production
  environment: production
  when: manual
  script:
    - ansible-playbook -i inventories/prod deploy.yml \
        -e tag=$CI_COMMIT_SHORT_SHA
```

Jenkins et GitLab CI : équivalences

Concept	Jenkins	GitLab CI
Configuration	Jenkinsfile	.gitlab-ci.yml
Exécution	Agent	Runner
Secrets	Credentials Jenkins	Variables CI/CD masquées
Approbation	input { }	when: manual + environnement protégé
Réutilisation	Shared Library	include et templates
Registre d'images	Docker Hub (plugin)	Container Registry intégré

Jenkinsfile : staging, approbation, production

```
stage('Deploy staging') {
  steps { sh './deploy.sh staging $TAG' }
}
stage('Tests de validation') {
  steps { sh './smoke-test.sh staging' }
}
stage('Approbation') {
  steps { input message: 'Déployer en prod ?' }
}
stage('Deploy prod') {
  steps { sh './deploy.sh prod $TAG' }
}
```



Validation humaine
Restreinte à un groupe

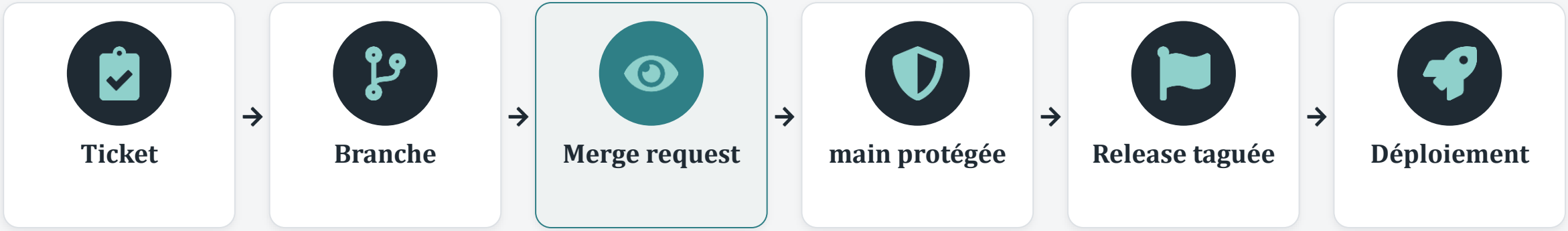


Tests de fumée
Avant la décision



Trace d'audit
Qui, quand, quelle version

Contrôle des changements




```
git commit -m "feat: endpoint paiement (#142)"

git tag -a v1.4.2 -m "Release 1.4.2"
git push origin v1.4.2
```

Commit lié au ticket, version sémantique, changelog.

Règles de protection

- Une approbation minimum avant fusion
- Pipeline vert obligatoire
- Environnement production protégé
- Traçabilité ticket, commit, déploiement

Blue-Green, Canary et feature flags

Blue-Green



Bascule du routeur, retour arrière instantané

Canary



On augmente la part de v2 tant que les métriques restent saines

Feature flags : le code est déployé mais la fonctionnalité reste désactivée. On l'active progressivement, et on la coupe sans redéployer.

Rollback automatisé

```
# smoke-test.sh : vérification après déploiement
if curl -fsS http://$HOST:8080/actuator/health; then
  echo "Déploiement validé"
else
  echo "Échec : retour à la version précédente"
  ansible-playbook -i inventories/prod deploy.yml \
    -e tag=$PREVIOUS_TAG
  exit 1
fi
```

```
post {
  failure {
    sh './scripts/rollback.sh $PREVIOUS_TAG'
  }
}
```

Conserver
le tag de la version précédente

Tester
la procédure en staging

Documenter
le runbook de retour arrière

Notifier
l'équipe en cas de rollback

Vérifier le service après la mise en production

Santé

```
curl -fsS http://IP:8080/actuator/health
```

Logs

```
docker logs --tail 100 backend
```

Ressources

```
docker stats --no-stream backend
```

Performance

```
k6 run --vus 20 --duration 30s load-test.js
```

Monitoring post-release : comparer latence et taux d'erreur avant et après déploiement, avec alertes sur seuils.

Piloter la CI en Agile et DevOps



Fréquence de déploiement
Nombre de mises en production par semaine



Délai de livraison
Du commit à la production



Taux d'échec
Part des déploiements qui échouent



Temps de restauration
Durée pour revenir à un service sain

Definition of Done
Pipeline vert, quality gate, documentation à jour, déployable.

Rétrospective
Analyser les pipelines en échec et les temps de build.

SBOM : l'inventaire des composants

```
# Générer le SBOM de l'image
syft scan docker.io/user/backend:1.0.0 \
  -o cyclonedx-json=sbom.cdx.json

# Rechercher des vulnérabilités connues
grype sbom:sbom.cdx.json
```

```
stage('SBOM') {
  steps {
    sh 'syft scan $IMAGE -o cyclonedx-json=sbom.cdx.json'
    archiveArtifacts artifacts: 'sbom.cdx.json'
  }
}
```



Traçabilité
Quels composants tiers dans quelle version



Vulnérabilités
Réagir vite quand une faille est publiée



Archivage
Un SBOM par version livrée

SLSA : sécuriser la chaîne de production

NIVEAU 1

Provenance

Le build produit une trace de sa provenance

NIVEAU 2

Build hébergé

Build sur une plateforme CI, provenance signée

NIVEAU 3

Build durci

Builds isolés, provenance non falsifiable

Signer et vérifier l'image

```
cosign sign --key cosign.key docker.io/user/backend:1.0.0
cosign verify --key cosign.pub docker.io/user/backend:1.0.0
```

On déploie uniquement une image dont la signature est vérifiée.

Rapport de conformité et audit trail

Pour chaque déploiement, conserver

- Commit et auteur
- Résultat Quality Gate
- Digest de l'image
- SBOM archivé
- Signature vérifiée
- Approbation : qui, quand

Rapport de conformité CI/CD

Architecture du pipeline · contrôles automatiques · procédure de mise en production · procédure de rollback · preuves d'exécution

Comité go / no-go et préparation du jury

GO si tous les critères sont réunis

- Quality Gate au vert
- Tests de validation réussis en staging
- SBOM généré et archivé
- Procédure de rollback testée
- Runbook et documentation à jour
- Approbation tracée

MSPR 5.1

Argumentaire technique

- Choix d'outils justifiés
- Déploiement fiable et réversible
- Preuves de conformité

Compétences travaillées : B5C3, B5C4 (E52) · du commit à la production, de façon fiable, documentée et réversible